

Master Specification

1. Product goal

FinWallet is a .NET 8 multi-currency digital-wallet side project intended to demonstrate real financial-backend concerns: transaction consistency, double-entry accounting, idempotency, concurrency, external integrations, fraud, authentication/session security, cutoff, campaigns, notifications, reconciliation, security and observability.

2. Current scope

****Customer/Auth****

- Registration with TR/AZ country/phone policy.
- Redis-backed OTP verification.
- PBKDF2 password hashing.
- JWT access token plus durable server session.
- Opaque refresh-token rotation and reuse detection.

****Wallet/Account****

- TRY/USD/EUR wallet create/list.
- External BankAccount opening through FakeBank.
- Wallet and BankAccount are separate domain concepts.
- No FX conversion in v1.

****Financial Core****

- Double-entry Ledger.
- Durable FinancialTransaction.
- MSSQL-backed durable idempotency.
- Atomic wallet-to-wallet transfer.
- Internal plus external fraud decision.

****Platform****

- YARP Gateway is the entry point for normal public/client and FinWallet-to-provider HTTP traffic.
- Gateway applies JWT, internal-service authorization, rate limiting, health checks and load balancing.
- Every Web API has Swagger; production defaults to disabled.

3. Not yet complete

- Public BankDeposit and BankWithdrawal.
- Merchant purchase / campaign accounting.
- Public Refund/Reversal flow.
- Durable FraudEvents/manual review.
- Transactional Outbox/Inbox.
- Transaction history/read model.
- ReconciliationRuns/ReconciliationIssues.

- Centralized masked structured logging, OpenTelemetry and alerting.
- Real MSSQL/Redis/YARP integration-concurrency test suite.
- Public logout/session-revoke endpoint.

4. Technology rules

- .NET 8 / C# 12.
- ASP.NET Core controller-based Web API.
- MSSQL as durable source of truth.
- Redis for transient state only.
- JWT auth; no ASP.NET Core Identity.
- YARP reverse proxy/gateway.
- Built-in DI.
- Paid/freemium NuGet packages are forbidden.
- Package versions are centrally managed in Directory.Packages.props.

5. Architecture

The main application is a modular monolith:

```
FinWallet.Api
FinWallet.Application
FinWallet.Domain
FinWallet.Infrastructure
FinWallet.Shared.Contracts
FinWallet.Shared.Web
FinWallet.Gateway
```

External simulators are separate Web API processes:

```
FakeBank.Api
FakeFraud.Api
FakeCutoff.Api
FakeCampaign.Api
FakeCommunication.Api
```

6. Financial-correctness invariants

- 1 No money movement may bypass the Ledger.
- 2 Every posted journal must satisfy total Debit = total Credit.
- 3 MSSQL is the final financial-consistency authority.
- 4 Redis loss must not create duplicate money, negative balance or ledger corruption.
- 5 External HTTP never runs inside an open financial SQL transaction.
- 6 Duplicate commands and retries must be safe.
- 7 Same idempotency key + different payload is a conflict.
- 8 Completed financial history is not mutated/deleted; corrections use reversal/compensation.
- 9 Currency is part of every Money value and validated before commit.
- 10 Reconciliation never silently repairs a mismatch with a balance UPDATE.

7. Gateway security rule

```
Client -> Gateway -> FinWallet.Api
FinWallet.Api -> Gateway /providers/* -> Fake providers
```

- Gateway requires JWT for protected public /api/* routes.
- FinWallet.Api repeats JWT and ownership validation.
- FinWallet.Api calls provider routes using InternalServiceKey.
- Gateway writes a separate DownstreamServiceKey on destination requests.
- Backend/provider business endpoints reject direct calls without the downstream key.

8. Financial-operation model

Wallet transfer order:

```
completed durable replay
-> durable session/risk signals
-> internal fraud
-> external fraud
-> final Allow
-> atomic MSSQL posting
```

Atomic posting commits idempotency + balances + FinancialTransaction + LedgerJournal + LedgerEntries in one transaction.

9. Security rules

- Never log password/OTP/JWT/refresh token/service keys/connection secrets.
- Use parameterized SQL.
- Authorization/ownership does not trust client-supplied identity or risk flags.
- JWT signing algorithm is a code-level invariant.
- PBKDF2 scheme changes require versioned migration.
- Rate/body/header/connection/time limits exist at Gateway and backend layers.
- Volumetric DDoS additionally requires ingress/WAF/cloud DDoS controls.

10. Definition of Done

A feature is complete only when:

- the solution builds successfully;
- relevant tests pass;
- financial/concurrency/idempotency invariants remain valid;
- external failure behavior is defined;
- security/logging impact is reviewed;
- package inventory is current;
- TR/EN XML documentation is current;
- all affected Markdown documents are TR+EN and consistent with code.

11. Explicitly out of scope for v1

- real bank/fraud/SMS/email providers;
- credit-card acquiring/payment gateway;
- full core banking;
- loans/credit scoring;
- stock/crypto trading;
- FX conversion;

- Event Sourcing;
- mandatory Kafka/RabbitMQ;
- full microservice decomposition.

Technical Analysis

1. Purpose

FinWallet's technical goal is to provide a financial-backend example in which money movements are correct, safely retryable, traceable and resilient to external-service failures. The system is designed not only for happy paths but also for duplicate requests, concurrency, provider timeouts, fraud failures, session revocation and data-consistency problems.

2. Functional scope

Currently implemented:

- customer registration + OTP verification;
- login, JWT access token, refresh-token rotation and server-side sessions;
- TRY/USD/EUR wallet create/list;
- external bank-account opening through FakeBank;
- internal/external fraud evaluation;
- durable idempotent wallet-to-wallet transfer;
- double-entry ledger;
- client and provider traffic through YARP Gateway;
- Swagger, rate limiting and shared HTTP security controls.

Planned but not yet complete:

- BankDeposit;
- BankWithdrawal;
- merchant purchase/campaign accounting;
- public refund/reversal flows;
- durable manual fraud review;
- outbox/inbox;
- reconciliation;
- transaction-history read model.

3. Critical quality requirements

****Financial correctness:**** Wallet balance, FinancialTransaction, IdempotencyRecord and Ledger changes must commit consistently inside the same financial transaction boundary.

****Idempotency:**** Retrying the same money-changing operation must not move money twice. Reusing the same key with a different payload must produce a conflict.

****Concurrency:**** Concurrent spending from one wallet must not cause overspending. MSSQL is the final authority.

****Security:**** Public traffic passes through Gateway JWT validation, while service-level JWT/ownership is validated again. Provider routes use separate internal and downstream service credentials.

****Availability:**** Provider timeout or 5xx responses must not corrupt financial correctness. Mandatory decisions such as fraud evaluation fail closed.

****Auditability:**** Financial movements must be explainable through ledger entries and immutable business-transaction records.

4. Data authorities

- **MSSQL:** durable source of truth for customer, authentication/session, wallet, bank account, transaction, idempotency and ledger state.
- **Redis:** transient state such as OTP challenges; never the authority for money correctness.
- **Fake providers:** simulate external systems and cannot directly mutate FinWallet tables.

5. Runtime boundaries

Client -> YARP Gateway -> FinWallet.Api
FinWallet.Api -> YARP Gateway -> Fake providers

The Gateway owns routing, edge security and traffic controls. Business authorization and financial rules remain in the application/domain layers.

6. Performance approach

- SqlConnection connection pooling is used.
- Redis ConnectionMultiplexer is singleton.
- HttpClientFactory + SocketsHttpHandler pooling is used.
- YARP destination connections, timeouts and load balancing are configuration-driven.
- SQL isolation/locking required for financial correctness is not weakened merely for benchmark throughput.

7. Testing approach

Moq is used only for Application orchestration boundaries. Real MSSQL locking, Redis Lua and YARP routing are not considered proven by mocks; they require integration/concurrency tests.

8. Most important current gap

A new wallet starts with zero balance and no public BankDeposit endpoint currently exists. Therefore a fully public-API-only register -> fund -> transfer path is not yet possible. This is the most important next functional gap in the project.

Architecture

1. Architectural style

The main financial application is kept as a **modular monolith**. The goal is to avoid turning Wallet, Transaction and Ledger components that participate in one atomic money movement into a distributed-transaction problem through premature microservice decomposition.

Main solution projects:

```
src/
  FinWallet.Api
  FinWallet.Application
  FinWallet.Domain
  FinWallet.Infrastructure
  FinWallet.Shared.Contracts
  FinWallet.Shared.Web
  FinWallet.Gateway
simulators/
  FakeBank.Api
  FakeFraud.Api
  FakeCutoff.Api
  FakeCampaign.Api
  FakeCommunication.Api
tests/
  FinWallet.Application.Tests
```

2. Layer responsibilities

Domain: entities, value objects, invariants, state transitions and financial rules. It has no knowledge of HTTP, SQL or provider DTOs.

Application: use-case orchestration and ports/interfaces. It uses Domain but does not depend on Infrastructure implementations.

Infrastructure: MSSQL, Redis, JWT token issuance, provider HttpClient adapters and persistence implementations.

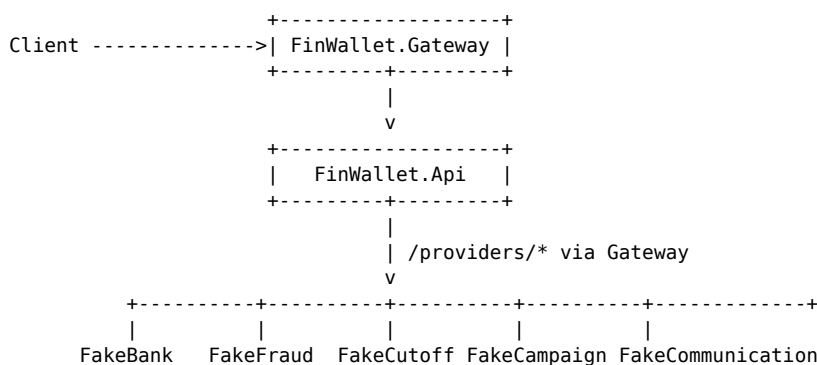
Api: controllers, HTTP-contract mapping, authentication/authorization composition and exception mapping.

Shared.Contracts: common HTTP envelope and stable transport contracts shared with simulators.

Shared.Web: shared hosting concerns such as Swagger, rate limiting, Kestrel limits, CORS, security headers and internal-service-key checks.

Gateway: YARP routes/clusters, edge JWT validation, internal-service authorization, load balancing and traffic controls.

3. Runtime topology



Clients normally do not call backend services directly. FinWallet.Api also does not call provider simulators directly; it uses Gateway provider routes.

4. Trust boundaries

- 1 ****Client -> Gateway:**** public authentication boundary. Protected /api/* routes require JWT.
- 2 ****Gateway -> FinWallet.Api:**** Gateway adds a separate downstream service credential. The API independently validates JWT and ownership.
- 3 ****FinWallet.Api -> Gateway provider route:**** requires InternalServiceKey.
- 4 ****Gateway -> simulator:**** requires DownstreamServiceKey.

This prevents a bypassed Gateway from automatically turning a backend business endpoint into a trusted endpoint.

5. Financial transaction boundary

A wallet-to-wallet transfer commits the following in one MSSQL transaction:

- durable idempotency;
- source-wallet debit;
- destination-wallet credit;
- FinancialTransaction;
- LedgerJournal;
- LedgerEntries.

External HTTP is never executed while that SQL transaction is open.

6. Dependency direction

```
Api -> Application -> Domain
Api -> Infrastructure -> Application/Domain
Gateway -> Shared.Web/Shared.Contracts
Simulators -> Shared.Web/Shared.Contracts
```

Domain must not depend on Infrastructure/API. Application must not know provider-specific DTOs or concrete SQL implementations.

7. Why not microservices?

Today's boundaries optimize for correctness and comprehensibility rather than independent deployment. Service decomposition can be reconsidered when domain boundaries are stable and there is a real need for independent scaling or team ownership. Splitting components such as Wallet and Ledger that require the same atomic commit is particularly expensive.

Design Patterns

FinWallet uses design patterns only where they solve a concrete financial problem, not to accumulate pattern names. Unnecessary abstraction or interface layers should be avoided.

DDD-lite

Entities, Value Objects, Aggregates and invariants describe financial-model boundaries. The project does not use full event sourcing or heavyweight DDD infrastructure.

Adapter + Anti-Corruption Layer

Application ports such as `IBankProvider`, `IExternalFraudProvider` and `ICommunicationGateway` keep provider-specific DTOs/enums out of Domain. FakeBank/FakeFraud details remain in Infrastructure.

Application Orchestrator

Use cases such as registration, bank-account opening and transfer are coordinated by handlers. Controllers do not own business workflows.

Chain of Responsibility / Rule Engine

Internal fraud evaluation is composed from independent `InternalFraudRule` rules. New rules can be added without substantially rewriting the transfer handler.

Policy Pattern

The combination of internal and external fraud decisions is explicitly represented through policy objects such as `FraudDecisionPolicy`.

Explicit Unit of Work / SQL Transaction Boundary

For wallet transfer, `SqlWalletTransferPostingStore` explicitly owns the transaction boundary instead of hiding it behind a generic `UnitOfWork` abstraction. Locking and commit order are part of business correctness.

Idempotent Command

Money-changing requests carry a client-generated `Idempotency-Key`. A durable MSSQL record safely replays the same key/same payload and rejects the same key/different payload.

Optimistic / Compare-and-Set Concurrency

Lifecycle updates such as `BankAccount` use expected status + timestamp CAS. High-contention money movement such as wallet transfer uses explicit locking/`Serializable` where required.

Double-Entry Ledger

Every financial effect is represented by a journal whose total Debit equals total Credit. Corrections use reversal/compensation rather than rewriting history.

State Machine

Authentication/session, `BankAccount` and `FinancialTransaction` lifecycles progress only through allowed state transitions.

Cache-Aside / TTL

Redis is used with TTL for transient state only. It is not a financial source of truth.

Timeout / Safe Retry

Provider calls have timeouts. Retries are allowed only when operation-level idempotency makes them safe; arbitrary financial POSTs are not automatically retried.

Transactional Outbox / Inbox

These are architectural plans but are not yet implemented in the production flow. They are intended for post-commit notifications and duplicate callback processing.

Saga / Compensation

This will be used only where long-running external-bank workflows justify it. A single-database wallet transfer does not use a saga because one MSSQL transaction is simpler and more correct.

Reconciliation

Ledger/balance/provider-statement mismatches are detected and investigated rather than silently corrected. Reconciliation infrastructure is planned.

CQRS-lite

Command/query code organization may be separated, but a separate read database or event-sourcing infrastructure is not required.

API Guide

Public entry point

Normal clients call only the Gateway:

```
http://localhost:8080
```

Main business prefix: /api/v1.

Anonymous Gateway routes:

- POST /api/v1/auth/register
- POST /api/v1/auth/registration/verify
- POST /api/v1/auth/login
- POST /api/v1/auth/refresh

Other /api/* routes require JWT at the Gateway. FinWallet.Api independently repeats JWT, authorization and ownership checks.

Common HTTP contract

All success/failure bodies use `ServiceResult<T>`. Client logic should branch on:

- 1 HTTP status;
- 2 stable code.

Human-readable message text must not be parsed for business logic.

Correlation

Clients may send X-Correlation-Id. The shared web platform accepts only bounded alphanumeric/-/_ values; invalid or absent values are replaced. Correlation IDs are not transaction or idempotency keys and must not contain PII.

Authentication

Protected endpoints use:

```
Authorization: Bearer <access-token>
```

JWT is validated at both Gateway and FinWallet.Api. Money-changing transfer also validates JWT sid against durable CustomerSession state.

Swagger

Development:

Gateway	http://localhost:8080/swagger
FinWallet.Api	http://localhost:8081/swagger
FakeBank.Api	http://localhost:8082/swagger
FakeFraud.Api	http://localhost:8083/swagger
FakeCutoff.Api	http://localhost:8084/swagger
FakeCampaign.Api	http://localhost:8085/swagger
FakeCommunication	http://localhost:8086/swagger

Swagger is disabled by default in production.

Authentication endpoints

****Register**** — POST /api/v1/auth/register Creates a pending customer and sends OTP through FakeCommunication. Success: HTTP 202.

****Verify**** — POST /api/v1/auth/registration/verify Verifies/consumes OTP and activates the customer.

****Login**** — POST /api/v1/auth/login Validates credentials and creates durable session, access token and refresh token.

****Refresh**** — POST /api/v1/auth/refresh Rotates an opaque single-use refresh token; reuse detection revokes the session family.

****Logout**** — not yet implemented as a public endpoint.

Wallet endpoints

****POST /api/v1/wallets**** — requires JWT.

```
{ "currency": "TRY" }
```

Supported currencies: TRY, USD, EUR. First create returns 201; repeating the same customer/currency returns the existing wallet with 200. New wallets start with zero available and blocked balance.

****GET /api/v1/wallets**** — lists wallets owned by the JWT subject.

Bank account

****POST /api/v1/bank-accounts**** — requires JWT.

```
{ "walletId": "<wallet-guid>" }
```

Flow:

```
Gateway JWT
-> API JWT + ownership
-> durable BankAccount(Opening)
-> SQL completes
-> Gateway /providers/bank/*
-> FakeBank
-> provider result validation
-> CAS state update
```

External HTTP never runs inside an open SQL transaction.

Wallet transfer

****POST /api/v1/transfers**** requires:

- JWT;
- active durable session;
- Idempotency-Key;
- owned/active source wallet;
- distinct active destination;
- same currency;
- positive/bounded amount;
- final fraud Allow;
- sufficient balance.

```
POST /api/v1/transfers
Authorization: Bearer <JWT>
Idempotency-Key: transfer-000001
Content-Type: application/json
```

```
{
  "sourceWalletId": "aaaaaaaa-1111-4111-8111-111111111111",
  "destinationWalletId": "bbbbbbbbb-2222-4222-8222-222222222222",
  "amount": 125.50
}
```

Order:

```
completed replay
-> session/risk
-> internal fraud
-> FakeFraud via Gateway
-> combined decision
-> atomic MSSQL posting
```

A completed replay returns the same immutable transaction without a second fraud evaluation or money movement.

Funding status

No public BankDeposit endpoint exists yet. Because a new wallet starts at zero, the source wallet cannot currently be funded after registration using public endpoints alone. Test fixtures must create a balanced FinancialTransaction + LedgerJournal/Entries rather than directly updating the wallet balance.

Internal/provider routes

```
/providers/bank/*
/providers/fraud/*
/providers/cutoff/*
/providers/campaign/*
/providers/communication/*
```

These are not public client APIs. Gateway requires internal-service authorization and destination business endpoints additionally validate the downstream service key.

Platform failure examples

- missing/invalid JWT: 401 GATEWAY_UNAUTHORIZED;
- direct backend without downstream key: 401 INTERNAL_SERVICE_UNAUTHORIZED;
- rate limit: 429 RATE_LIMITED;
- unsupported write content type: 415;
- TRACE/CONNECT: 405;
- body/header limits: rejected before business processing.

External Integrations

Integration principles

FinWallet treats every simulator as an external provider even though simulator source code lives in the same repository.

Rules:

- provider services are controller-based ASP.NET Core APIs;
- success/failure bodies use `ServiceResult<T>`;
- FinWallet never reads or writes provider storage directly;
- provider DTOs/enums stay behind Infrastructure anti-corruption adapters;
- internal and provider identifiers remain separate;
- external HTTP does not run while a financial SQL transaction is open;
- retries are used only when operation-level idempotency makes them safe;
- normal provider calls pass through YARP Gateway.

Gateway provider routes

```
/providers/bank/  
/providers/fraud/  
/providers/cutoff/  
/providers/campaign/  
/providers/communication/
```

Local default: `http://localhost:8080/providers/....`

FinWallet.Api calls the Gateway with `X-Internal-Service-Key`. Gateway validates that credential and uses a separate `DownstreamServiceKey` on the destination request. Provider business endpoints reject direct calls without the downstream key.

HttpClient policy

Provider adapters use `HttpClientFactory` + `SocketsHttpHandler` with:

- provider-specific timeout;
- max connections per server;
- pooled connection lifetime/idle timeout;
- GZip/Deflate/Brotli decompression;
- cookies disabled;
- internal-service-header delegating handler.

Arbitrary financial POSTs are not automatically retried because a timeout does not prove that the remote side effect did not happen.

FakeCommunication.Api

Simulates SMS/communication. Primary endpoint:

```
POST /api/v1/communication/sms
```

FinWallet route:

```
POST /providers/communication/api/v1/communication/sms
```

OTP bodies are sensitive and must not be logged in production. Fake modes can simulate failure, delay and timeout.

FakeBank.Api

Owns provider-side account/transaction state and never mutates FinWallet Wallet/Ledger tables.

Supported provider concepts include:

- account opening/read/activation/pending;
- deposit/withdrawal-like transaction start/finalize/read;
- statement data for reconciliation;
- duplicate protection through stable provider request keys.

IBankProvider is the Application port and FakeBankProvider the Infrastructure adapter. The adapter maps envelopes/enums/currency, propagates correlation and does not perform unsafe automatic financial retries.

FakeFraud.Api

Independent external-fraud simulator.

```
POST /providers/fraud/api/v1/fraud/evaluate
```

Requests contain server-derived risk signals. Raw device ID is replaced with a stable hashed reference. External fraud is mandatory for transfer when required; timeout/network/malformed responses fail closed.

FakeCutoff.Api

Simulates business-calendar/cutoff/settlement calculations by bank, currency and transaction type. It is mainly planned for BankDeposit and BankWithdrawal. Simulated holiday data is not a legal production source.

FakeCampaign.Api

May calculate campaign eligibility, discount and sponsor identity. FinWallet remains responsible for representing the economic effect in a balanced ledger.

Health and load balancing

YARP clusters use configuration-driven destinations and PowerOfTwoChoices. Critical provider clusters use active health checks; the main FinWallet cluster also has passive transport-failure health handling.

SSRF / unsafe upstream prevention

Clients cannot provide arbitrary provider URLs. Provider destinations are server-owned YARP/config values, so normal request payloads cannot turn the application into an HTTP client for user-controlled URLs.

Production network rule

Application-level downstream keys reduce direct-bypass risk, but production backend/provider services should additionally be non-publicly-routable through network policy/ingress controls.

Technologies and NuGet Packages

Technology baseline

Technology	Version/Policy	Purpose
.NET	8	Runtime and application platform
ASP.NET Core	8	Controller-based Web API
C#	12	Application language
YARP	2.3.0	Reverse proxy, routing, health, load balancing
Swashbuckle	10.2.3	Swagger/OpenAPI generation and UI
Microsoft.Data.SqlClient	7.0.2	MSSQL durable persistence
StackExchange.Redis	3.0.17	Transient Redis state
JwtBearer	8.0.29	Gateway/API JWT validation
xUnit v3	3.2.2	Unit-test framework
Moq	4.20.72	Application-boundary mocks

Package policy

Paid/freemium NuGet dependencies are not allowed. Selection order:

- 1 built-in .NET/ASP.NET Core capability;
- 2 an appropriate Microsoft-maintained package;
- 3 only fully free/open-source third-party packages with compatible licensing.

All versions are centrally pinned in Directory.Packages.props.

JwtBearer

Used for JWT validation at Gateway and FinWallet.Api. Hand-written JWT parsing/signing is avoided because it introduces unnecessary security risk. The signing algorithm remains a code-level invariant.

Microsoft.Data.SqlClient

Used for explicit async SQL, transactions, locking and financial persistence. MSSQL is the durable financial authority. Financial SQL values must be parameterized. Generic ORM/repository abstractions must not hide important locking behavior.

StackExchange.Redis

Used for OTP and other transient distributed state. ConnectionMultiplexer is singleton. Redis never becomes the Wallet/Ledger/idempotency financial source of truth.

Yarp.ReverseProxy

Used for Gateway route matching, authorization policies, transforms, cluster load balancing, health checks and transport tuning. It does not contain financial business rules.

Swashbuckle.AspNetCore

Provides common Swagger generation/UI across all Web APIs through FinWallet.Shared.Web. It is disabled by default in production.

xUnit v3 + Moq

Used for Application orchestration unit tests. Moq is appropriate for isolating external-provider/persistence boundary calls, but does not prove real MSSQL locking, Redis Lua or YARP routing behavior.

Framework-only capabilities

No separate package is added for:

- Kestrel limits;
- ASP.NET Core rate limiting;
- CORS;
- security-header middleware;
- HttpClientFactory/SocketsHttpHandler;
- PBKDF2/HMAC/SHA cryptography;
- CancellationToken/TimeProvider.

PBKDF2 note

PBKDF2 V1 work factor is not a loose appsettings tuning value. Existing credential rows do not store per-password iteration metadata, so changing it requires a versioned hash migration/rehash design.

New-package approval record

Every future package must document exact ID, version, license, owner project, purpose, why built-in functionality is insufficient, considered alternatives and financial/security impact.

Database Design

Source of truth

MSSQL is the durable source of truth for customer, authentication/session and financial state. Redis is not a replacement for MSSQL and cannot independently guarantee money correctness.

Persistence uses explicit parameterized Microsoft.Data.SqlClient. SQL transaction boundaries remain visible, especially in financial locking code.

Authentication schema

database/001_authentication_schema.sql:

- Customers;
- CustomerCredentials;
- CustomerSessions;
- RefreshTokens.

Raw passwords and raw refresh tokens are never stored.

Financial accounts schema

database/002_financial_accounts_schema.sql:

- Wallets;
- BankAccounts.

Wallet invariants:

- unique (CustomerId, Currency);
- non-negative available/blocked balances;
- customer ownership FK;
- durable lifecycle;
- DECIMAL(19,4) balance.

BankAccount invariants:

- one BankAccount per Wallet;
- internal and provider AccountId remain separate;
- composite wallet/customer/currency relationship;
- provider account ID and IBAN-like value remain consistent;
- lifecycle updates use expected status + UpdatedAt compare-and-set.

Ledger/transaction schema

database/003_ledger_transaction_schema.sql defines:

- LedgerAccounts;
- FinancialTransactions;
- IdempotencyRecords;
- LedgerJournals;
- LedgerEntries.

A **FinancialTransaction** is the durable business-operation record; it is not the same concept as a Wallet row or LedgerJournal.

Stable transaction types:

- 1 WalletTransfer
- 2 BankDeposit
- 3 BankWithdrawal
- 4 Refund
- 5 Reversal

Lifecycle:

- 1 Processing
- 2 Completed
- 3 Failed
- 4 Reversed

Financial decimal rule

Financial amounts must fit DECIMAL(19,4):

- maximum four decimal places;
- maximum absolute amount 999999999999999.9999.

Application boundaries validate amount/balance capacity before SQL rather than relying on database overflow errors.

Durable idempotency

Identity:

Scope + CustomerId + IdempotencyKey

Wallet-transfer scope: WALLET_TRANSFER.

The request fingerprint is SHA-256 over canonical source/destination/amount values.

Behavior:

- same key + same payload -> wait/replay the same transaction;
- same key + different payload -> conflict;
- MSSQL is final authority;
- replay returns immutable transaction fields, not current wallet balances.

The posting store uses Serializable isolation with UPDLOCK, HOLDLOCK for the idempotency key range.

Wallet-transfer accounting

Wallet Liability ledger account code:

WALLET-LIABILITY:{walletId:N}

Accounting:

Debit	source wallet liability	amount
Credit	destination wallet liability	amount

This is a pure internal liability reclassification.

Atomic posting — implemented

SqlWalletTransferPostingStore performs in one MSSQL transaction:

```
BEGIN SERIALIZABLE
-> claim/replay idempotency
-> wallet row locks (deterministic GUID order)
-> validate ownership/status/currency/balance
-> calculate domain debit/credit
-> find/create ledger accounts
-> insert Processing FinancialTransaction
-> create/post LedgerJournal + entries
-> SQL aggregate Debit/Credit validation
-> update wallet balances
-> finalize transaction
-> finalize idempotency
-> COMMIT
```

Any exception before commit rolls back the complete financial state.

Debit = Credit defense in depth

Domain LedgerJournal.Post(...) validates balance. Persistence also aggregates inserted entries in SQL. Commit is forbidden unless there are at least two entries, positive totals and exact Debit/Credit equality.

Domain rehydration

Persisted entity state is materialized through controlled Restore factories rather than reflection-based mutation of private setters.

Redis state

Redis stores transient challenge state such as registration OTP. Wallet/Ledger/FinancialTransaction truth never moves to Redis.

Current status

Public POST /api/v1/transfers is **implemented** and uses the posting store through Application fraud/session orchestration. The previous documentation statement that no public transfer endpoint existed is obsolete and has been removed.

Remaining database work

- BankDeposit/BankWithdrawal persistence workflow;
- FraudEvents/manual review;
- Outbox/Inbox;
- ReconciliationRuns/ReconciliationIssues;
- AuditEvents/operational logging storage approach;
- integration/concurrency test fixtures.

Financial Flows

General rule

No money movement may bypass the Ledger. Wallet current balance is operational state; Ledger and FinancialTransaction records provide the explainable financial history.

1. Wallet-to-Wallet Transfer — implemented

Endpoint: POST /api/v1/transfers.

Flow:

```
Gateway JWT
-> API JWT + durable session
-> completed idempotency replay
-> server-side risk signals
-> internal fraud
-> external FakeFraud via Gateway
-> final Allow
-> one MSSQL financial transaction
    -> idempotency
    -> source debit
    -> destination credit
    -> FinancialTransaction
    -> LedgerJournal + LedgerEntries
-> commit
```

Accounting:

```
Debit    Source Wallet Liability
Credit   Destination Wallet Liability
```

Replaying the same request/key never moves money a second time.

2. BankDeposit — planned next flow

The purpose is to fund a FinWallet wallet after an external bank movement has been confirmed. No public endpoint exists yet.

Proposed lifecycle:

```
Created
-> Fraud/Cutoff if required
-> BankPending
-> BankConfirmed
-> LedgerPost
-> Completed
```

The wallet must not be credited before external confirmation. Deposit posting should create a balanced journal between a provider-side asset/clearing account and the customer wallet liability.

3. BankWithdrawal — planned

Withdrawal is a long-running external workflow, so available funds should first move into blocked state.

Proposed state:

```
Available -> Blocked -> external bank pending
                -> success: settle
                -> failure: release/compensate
```

The SQL transaction must not remain open during external HTTP.

4. Refund / Reversal — planned public flow

Refund is a business-level return, while reversal is the accounting inverse of a previous financial effect. The original transaction/journal is not deleted or mutated. A new FinancialTransaction and reverse

journal are created.

5. Campaign / Merchant Purchase — planned

FakeCampaign may calculate eligibility and discount, but FinWallet Ledger remains responsible for correctly allocating the economic effect across customer, merchant and sponsor accounts.

6. Fraud Review

A Review decision currently moves no money. Durable FraudEvents/manual-review queueing is not implemented yet.

7. Cutoff

FakeCutoff evaluates processing/settlement eligibility by bank, currency and transaction type. This integration is mainly planned for BankDeposit and BankWithdrawal.

8. Notification

Notification failure after a financial commit must not reverse the money transaction. Reliable post-commit notification is therefore planned around an Outbox pattern.

9. Reconciliation

Planned checks include:

- wallet current state vs ledger-derived balance;
- internal bank-related transactions vs FakeBank statement;
- orphaned idempotency/transaction/ledger records.

Mismatches are recorded and investigated rather than silently fixed with an UPDATE.

10. Current demo limitation

New wallets start at zero balance. Because public BankDeposit does not yet exist, a new customer cannot currently be fully funded and transferred using only public endpoints. An integration fixture must atomically create balance + FinancialTransaction + balanced LedgerJournal/Entries; directly updating Wallets.AvailableBalance is invalid.

Security Design

Core principles

- Financial correctness, least privilege and credential safety take priority.
- Normal HTTP traffic passes through YARP Gateway.
- Gateway authentication does not replace service/domain authorization.
- MSSQL is the durable authentication/financial source of truth.
- Redis is transient support state.
- Production secrets are not committed and startup fails fast when required secrets are absent.
- Passwords, OTPs, JWTs, refresh tokens, service keys and connection secrets are never logged.
- Ownership/risk facts are server-derived.

Gateway trust model

****Client -> Gateway:**** protected /api/* routes require JWT; register/verify/login/refresh are anonymous.

****Gateway -> FinWallet.Api:**** Gateway supplies a downstream service key; the API independently revalidates JWT and ownership.

****FinWallet.Api -> Gateway /providers/*:**** requires InternalServiceKey.

****Gateway -> Fake provider:**** requires a separate DownstreamServiceKey.

Health routes remain open for orchestrator/YARP checks; Swagger exposure is environment-configured.

HTTP attack surface

FinWallet.Shared.Web applies:

- Kestrel server header disabled;
- TRACE/CONNECT blocked;
- JSON required for body-bearing POST/PUT/PATCH;
- body/header-count/header-size limits;
- request-header/keep-alive timeouts;
- max concurrent connections;
- per-IP fixed-window rate limit;
- zero queue by default;
- CORS allow-list;
- bounded correlation ID;
- no-store/no-cache;
- CSP plus frame/MIME/referrer/permissions/cross-origin headers.

These controls reduce L7 abuse/resource exhaustion; volumetric DDoS still requires cloud/edge/ingress/WAF protections.

Passwords

V1:

- PBKDF2-HMAC-SHA512;

- 220,000 iterations;
- 32-byte random salt;
- 64-byte hash;
- constant-time comparison;
- persisted hash version.

Iteration count is not a loose runtime tuning value; future change requires versioned migration/rehash.

JWT and sessions

JWT uses fixed HMAC-SHA256 with minimal sub, sid, JTI and iat claims. It contains no balance/IBAN/contact data. Issuer/audience/signing key come from configuration/secret store. Lifetime is configurable only within a safe range. Signing algorithm is not configurable.

High-risk transfer validates sid against MSSQL CustomerSession state so a revoked session can stop money movement before its short-lived JWT expires.

Refresh tokens

Clients receive opaque random tokens; raw tokens are not persisted. Rotation is single-use through compare-and-set semantics. Reuse of a consumed token revokes the session/token family.

OTP

Redis stores an HMAC digest rather than raw OTP. Lua scripts provide atomic issue/attempt/consume behavior. Verification fails closed when Redis is unavailable.

SQL injection

Financial/auth persistence uses parameterized SqlClient commands. Request text is never used as a dynamic SQL fragment.

BOLA / ownership

- customer identity comes from JWT sub;
- owned-resource queries include customer boundaries;
- transfer source ownership is validated before fraud and again inside locked posting;
- destination/currency/lifecycle are revalidated before commit.

Fraud security

Clients cannot supply trust flags such as isNewDevice, known beneficiary, velocity or 24-hour amount. Signals come from durable server state.

```
completed replay
-> server-side risk signals
-> internal fraud
-> external fraud
-> combined decision
-> posting only on Allow
```

External fraud unavailable/malformed => fail closed.

Idempotency / replay

Money-changing transfer requires Idempotency-Key. Durable identity is Scope + CustomerId + Key; a canonical request hash turns same-key/different-payload reuse into a conflict.

Logging

Never log:

- password/hash/salt;
- OTP/digest/pepper;
- JWT/refresh token;
- Authorization header;
- internal/downstream keys;
- signing key;
- SQL/Redis connection strings containing credentials;
- unmasked phone/email/IBAN/account values.

OWASP/API security summary

- Broken Access Control/BOLA: gateway + service auth and owner-aware SQL.
- Authentication failures: JWT/session/refresh rotation/login lockout/OTP/rate limit.
- Injection: parameterized SQL.
- Security Misconfiguration: prod Swagger off, bounded HTTP config, no server header.
- Cryptographic failures: PBKDF2, HMAC, short JWT, secret management.
- Sensitive business flows: durable idempotency, fraud and ledger.
- Resource consumption: rate/body/header/connection/timeouts.
- SSRF/unsafe API consumption: provider URLs are server-owned configuration.
- Supply chain: central package versions and CI build/test; SBOM/vulnerability scanning remains open work.

Remaining security work

- Gateway bypass/rate-limit integration tests;
- dependency vulnerability/SBOM scanning;
- centralized masked logging/SIEM/alerting;
- NetworkPolicy/ingress/TLS hardening;
- logout/session-revoke endpoint;
- durable FraudEvents/manual review;
- incident/reconciliation runbooks.

Testing Strategy

Current status

Before Gateway/platform hardening, the solution had no test project and no Moq usage. It now contains tests/FinWallet.Application.Tests using:

- xUnit v3;
- Moq;
- .NET 8 / Microsoft Testing Platform.

The first strict-mock test targets OpenBankAccountHandler, mocks IWalletStore, IBankAccountStore and IBankProvider, and proves that the external bank provider is not called when the owned wallet is missing.

What should be mocked?

Application orchestration tests may mock boundaries whose behavior is not the subject of the test:

- IBankProvider;
- IExternalFraudProvider;
- ICommunicationGateway;
- persistence interfaces;
- TimeProvider for time-dependent behavior.

Strict mocks are preferred for financial orchestration because an unexpected provider/database call may be a meaningful defect.

What cannot be proven with mocks?

The following require real infrastructure integration/concurrency tests:

- MSSQL rollback;
- Serializable / UPDLOCK / HOLDLOCK;
- unique-key races;
- refresh-token compare-and-set;
- deterministic wallet lock ordering;
- idempotency range locking;
- persisted Debit=Credit aggregate validation;
- connection-pooling behavior;
- Redis Lua atomicity/reconnect/fail-closed behavior;
- YARP routing/auth/health/load balancing;
- Gateway direct-bypass rejection.

Required integration suite

At minimum:

- 1 concurrent duplicate registration;
- 2 concurrent refresh rotation;
- 3 ten transfers with the same Idempotency-Key;

- 4 same key + different payload;
- 5 one hundred concurrent debits from one source wallet;
- 6 opposite-direction A->B and B->A deadlock test;
- 7 Redis unavailable during OTP verify;
- 8 FakeFraud timeout before transfer posting;
- 9 Gateway request without JWT;
- 10 backend request without downstream key;
- 11 rate-limit rejection;
- 12 ledger imbalance injection -> rollback.

Financial invariant test

Load tests must not trust HTTP success counts alone:

```
Initial balances
+ durable credits
- durable debits
= final balances
```

```
SUM(Ledger Debit) = SUM(Ledger Credit)
```

Successful FinancialTransactions, IdempotencyRecords and LedgerJournals must also reconcile.

CI policy

CI runs:

```
restore
-> Release build --warnaserror
-> dotnet test --no-build
```

A new project is not considered complete until it is included in FinWallet.sln.

Current coverage assessment

Unit-test infrastructure now exists, but coverage is still at an early stage. “Moq is present” does not mean real MSSQL/Redis/YARP correctness has been proven.

Code Documentation Standard

...

Code snippets, endpoint path'leri ve teknik identifier'lar çevrilmeden kullanılabilir; açıklama metni iki dilde verilmelidir.

Review checklist

Değişiklik tamamlanmamıştır eğer:

- gerekli declaration'da TR/EN XML doc yoksa;
- açıklama sadece member adını tekrar ediyorsa;
- financial side effect/invariant gizliyse;
- code davranışı değiştiği halde Markdown eski davranışı anlatıyorsa;
- yalnız bir dil güncellenmişse.

English

Purpose

FinWallet keeps C# XML documentation bilingual so business intent, financial invariants and technical behavior remain understandable in both Turkish and English.

Mandatory scope

Document every applicable manually written declaration:

- class/record/struct/enum/interface;
- constructor;
- method;
- property;
- generic type parameter;
- method parameter;
- return value;
- meaningful thrown exception.

Generated code is excluded.

Format

Every XML summary contains Turkish first and English second:

/// <summary> /// TR: Cüzdanın kullanılabilir bakiyesini temsil eder. /// EN: Represents the wallet's available balance. /// </summary>

Method/param/returns documentation should explain intent and constraints instead of merely repeating member names.

Financial-code expectations

Where relevant, documentation should explain:

- currency consistency;
- Debit = Credit;
- append-only/reversal behavior;
- idempotency;
- concurrency/locking assumptions;
- allowed state transitions;
- source-of-truth ownership;
- external HTTP / transaction boundaries.

Compiler enforcement

`Directory.Build.props` enables XML documentation generation and treats `CS1591` as an error. This provides compiler enforcement for externally visible members. Review standards extend useful documentation expectations to internal/private production members.

Markdown standard

`docs/\*\*/\*.md` and root `README.md` must be Turkish + English. Preferred structure:

Türkçe

...

English

...

Code snippets, endpoint paths and technical identifiers may remain unchanged; explanatory prose should exist in both languages.

Review checklist

A change is not complete when:

- a required declaration lacks TR/EN XML docs;
- documentation only repeats the member name;
- financial side effects/invariants are hidden;
- code behavior changed while Markdown still describes the old behavior;
- only one language was updated.

Agent and Codex Development Workflow

Purpose

This document defines how Codex or other coding agents scope work, create branches, validate changes, open pull requests and participate in review for FinWallet. Repository files, AGENTS.md, documentation, ADRs and GitHub issues/PRs are the source of truth; chat history does not replace repository documentation.

Operating model

Each agent:

- 1 receives a bounded task/issue;
- 2 reads repository rules;
- 3 creates an agent/<task> branch from current main;
- 4 makes required code/test/document changes;
- 5 validates with Release build/tests;
- 6 opens a reviewable pull request.

Recommended roles:

- Solution Architect;
- Security/Auth;
- Financial Domain;
- Persistence/Concurrency;
- Integration;
- QA/Chaos;
- Code Review;
- Documentation.

Source-of-truth order

When instructions conflict:

- 1 current task acceptance criteria;
- 2 AGENTS.md;
- 3 master specification;
- 4 accepted ADR;
- 5 architecture/security/database/API docs;
- 6 existing implementation conventions;
- 7 agent assumptions.

An agent must not silently override a higher-level rule.

Branch/commit

Branch format: agent/<bounded-task-name>.

Commits should be small and cohesive. Avoid vague commits such as implement everything or fix stuff. Financial behavior, package changes, refactors and documentation should remain independently

understandable when practical.

Coding-agent responsibilities

- preserve financial invariants;
- write required TR/EN XML docs for classes/interfaces/methods/properties;
- do not add paid/freemium dependencies;
- update package inventory for new packages;
- add/update tests for changed behavior;
- update API/architecture/database/security docs;
- never place external HTTP inside a financial SQL transaction;
- preserve idempotency/concurrency guarantees;
- never make Redis the financial source of truth;
- keep TR+EN Markdown documentation synchronized.

Review-agent checks

- architecture dependency violations;
- ledger/financial correctness;
- double-spend/lost updates;
- idempotency gaps;
- auth/OTP/token weaknesses;
- sensitive logging;
- unsafe retry/timeout behavior;
- missing cancellation;
- over-engineering;
- undocumented packages;
- missing TR/EN code/documentation.

QA/Chaos scenarios

- simultaneous overspend;
- repeated idempotency key;
- same key + altered payload;
- Redis outage;
- provider timeout/500/slow response;
- fraud unavailable;
- repeated refund/reversal;
- ledger mismatch;
- refresh-token replay;
- OTP brute force;
- direct Gateway bypass;
- rate-limit/resource exhaustion.

Pull-request flow

- 1 Complete bounded work on the branch.
- 2 Self-review the diff.
- 3 Update docs/tests.
- 4 Run dotnet restore, Release dotnet build --warnaserror, and dotnet test.
- 5 Open a draft PR.
- 6 Resolve review findings with small commits.
- 7 CI becomes green.
- 8 Financial/security-sensitive PRs receive final review.
- 9 Merge only after acceptance criteria are satisfied.

Parallel-agent rule

Until domain boundaries, DB/ledger rules, API contracts, auth and error/idempotency conventions are stable, parallel agents should not invent incompatible contracts inside the same feature area. Parallel work is appropriate when file overlap is low and contracts are stable.

DoD

Agent-generated code is done only when acceptance criteria, build, relevant tests, financial/security invariants, TR/EN XML docs, package inventory and affected TR/EN Markdown docs are complete.

Project Management and Delivery Roadmap

Purpose

GitHub issues are authoritative work items. Implementation occurs on agent/* branches and is integrated into main through pull requests.

Board model

Recommended columns:

- 1 Backlog
- 2 Ready
- 3 In Progress
- 4 In Review
- 5 Blocked
- 6 Done

Equivalent status:* labels may be used until a Project board is attached.

Milestones

Milestone	Scope	Current status
M0 Architecture Baseline	scope, architecture, security, docs	Largely complete
M1 Foundation	solution/domain foundations	Complete
M2 Identity & Registration	OTP, password, JWT, refresh/session	Main flow complete; logout open
M3 Persistence & Concurrency	MSSQL, Redis, idempotency	Core transfer baseline complete; outbox/inbox open
M4 Financial Core	ledger, transaction, reversal model	Ledger + transfer complete; public reversal/refund open
M5 External Providers	FakeBank/Fraud/Cutoff/Campaign/Communication	Simulator baseline complete
M6 Financial Flows	deposit, withdrawal, transfer, purchase, refund	Wallet transfer complete; others open
M7 Reconciliation & Hardening	reconciliation, observability, chaos/integration tests	Open
M8 Gateway/Platform	YARP, Swagger, rate limit, security baseline	Complete

Major completed work

- registration/auth/session baseline;
- Wallet/BankAccount persistence;
- Fake provider APIs;
- double-entry ledger schema/domain;
- durable idempotent wallet transfer;
- fraud orchestration;
- YARP Gateway + downstream trust boundary;

- Swagger across all Web APIs;
- appsettings-based platform/performance tuning;
- initial xUnit v3 + Moq test project;
- Release build + test CI;
- bilingual documentation standard.

Open priorities

- 1 BankDeposit.
- 2 BankWithdrawal + cutoff.
- 3 Real MSSQL/Redis/YARP integration/concurrency tests.
- 4 Outbox/Inbox + reliable notification.
- 5 Durable FraudEvents/manual review.
- 6 Transaction history/read model.
- 7 Public Refund/Reversal flows.
- 8 Reconciliation.
- 9 Centralized masked logging/OpenTelemetry/alerting.
- 10 Production deployment hardening.

Roles

- Solution Architect: boundaries/ADR/dependency direction.
- Security/Auth: auth/session/OTP/token controls.
- Financial Domain: wallet/ledger/transaction/accounting.
- Persistence/Concurrency: SQL/Redis/idempotency/locking.
- Integration: provider adapters/resilience/contracts.
- QA/Chaos: concurrency/duplication/failure scenarios.
- Code Review: architecture/security/financial correctness.
- Documentation: TR+EN docs and code-doc synchronization.

Pull-request workflow

- 1 Define issue/acceptance criteria.
- 2 Create a bounded branch from current main.
- 3 Update code + tests + docs within the same feature scope.
- 4 Run Release build/tests.
- 5 Open a draft PR.
- 6 Resolve review/CI findings.
- 7 Merge after DoD is satisfied.

DoD

An issue is not Done until code, tests, TR/EN XML docs, affected TR/EN Markdown docs, security impact, failure behavior, concurrency/idempotency considerations and package inventory are complete.

Wallet Transfer Flow

Endpoint

POST /api/v1/transfers

Requirements:

- valid JWT;
- active durable session matching JWT sid;
- Idempotency-Key;
- source wallet owned by JWT sub customer;
- distinct destination wallet;
- positive DECIMAL(19,4)-compatible amount;
- matching source/destination currency;
- final fraud Allow;
- sufficient source balance.

```
{
  "sourceWalletId": "11111111-1111-1111-1111-111111111111",
  "destinationWalletId": "22222222-2222-2222-2222-222222222222",
  "amount": 125.50
}
```

Currency is not trusted from the client; it is derived from wallet state.

Idempotency order

Completed replay is checked ****before**** fraud:

```
request
-> completed replay lookup
  -> same key + same immutable payload: original result
  -> same key + different payload: conflict
-> durable session/risk
-> internal fraud
-> external fraud
-> atomic posting
```

This precheck is a semantic/performance guard. Final correctness still belongs to Serializable idempotency locking inside the atomic posting store.

Session validation

A valid JWT alone is not sufficient for money movement. sid must:

- belong to the same customer;
- not be revoked;
- not be expired;
- belong to an Active customer session.

Server-derived fraud signals

The client does not submit risk flags. SqlWalletTransferRiskSignalStore derives from server state:

- customer country;
- wallet currency;

- device reference/history;
- new-device flag;
- five-minute transfer velocity;
- 24-hour same-currency amount;
- known beneficiary.

Raw DeviceId is not sent to the provider; a stable SHA-256 reference is used.

Fraud flow

Internal Deny cannot be overridden by external Allow. FakeFraud is required after internal Allow/Review. External timeout/network/malformed response fails closed.

Internal + External -> FraudDecisionPolicy -> Allow / Review / Deny

- Allow -> posting;
- Review -> no money movement;
- Deny -> no money movement;
- provider unavailable -> no money movement.

A durable manual-review queue is not yet implemented.

Atomic posting

After final Allow, one MSSQL transaction commits:

- IdempotencyRecord;
- source debit;
- destination credit;
- FinancialTransaction;
- Wallet Liability LedgerAccounts;
- balanced LedgerJournal + LedgerEntries;
- persisted SQL Debit/Credit equality validation.

External HTTP never runs inside this transaction.

Double-entry

Debit	source	wallet	liability	amount
Credit	destination	wallet	liability	amount

Journal balance is validated at both Domain and SQL levels.

Replay response

Replay contains only immutable fields:

- FinancialTransaction ID;
- source/destination wallet IDs;
- amount/currency;
- original completion time;
- replay flag.

Current wallet balances are excluded because they may have changed since the original transaction.

Gateway, Swagger and Platform Security

Topology

```
Client
-> FinWallet.Gateway :8080
  -> FinWallet.Api :8081
    -> Gateway /providers/bank/*           -> FakeBank.Api :8082
    -> Gateway /providers/fraud/*          -> FakeFraud.Api :8083
    -> Gateway /providers/cutoff/*         -> FakeCutoff.Api :8084
    -> Gateway /providers/campaign/*       -> FakeCampaign.Api :8085
    -> Gateway /providers/communication/* -> FakeCommunication.Api :8086
```

FinWallet does not call provider simulators directly; provider base URLs point back to the Gateway.

Customer authentication boundary

Only register/verify/login/refresh are anonymous at the Gateway. Other /api/* routes require JWT through GatewayAuthenticated. FinWallet.Api independently repeats JWT and business-ownership validation.

Internal provider boundary

FinWallet.Api sends X-Internal-Service-Key to /providers/*. Gateway validates it through InternalService and writes a separate DownstreamServiceKey to the proxied request. Destination business endpoints return 401 without the downstream key.

This creates two separate trust boundaries:

- 1 FinWallet -> Gateway;
- 2 Gateway -> destination.

YARP routing/load balancing

Cluster configuration lives in appsettings. PowerOfTwoChoices, destination addresses, health intervals/timeouts, passive reactivation, max connections and activity timeout can be overridden without source-code changes. Development may use one destination while production can add replicas.

Rate limiting / resource exhaustion

Repository-level controls:

- Gateway per-IP fixed-window rate limit;
- backend second-layer rate limit;
- zero queue by default;
- request body/header-count/header-byte limits;
- request-header/keep-alive timeouts;
- max concurrent connections;
- provider-specific YARP body limits;
- provider HttpClient timeout/connection limits.

These are L7 overload/abuse controls. Volumetric DDoS still requires cloud DDoS protection, L4/L7 load balancing, WAF/CDN/bot protection and network policy.

Swagger

Swagger is shared through FinWallet.Shared.Web across:

- Gateway;

- FinWallet.Api;
- FakeBank;
- FakeFraud;
- FakeCutoff;
- FakeCampaign;
- FakeCommunication.

It is enabled in development and disabled by default in production. Swagger visibility does not bypass endpoint authorization.

Shared HTTP security baseline

- Kestrel Server header off;
- TRACE/CONNECT blocked;
- JSON write-content enforcement;
- correlation ID validation/regeneration;
- Cache-Control: no-store;
- X-Content-Type-Options: nosniff;
- X-Frame-Options: DENY;
- restrictive Referrer/Permissions/Cross-Origin policies;
- CSP;
- CORS allow-list;
- rate limiting;
- request/header/body/connection bounds.

OWASP/API mapping

****Broken Access Control/BOLA:**** Gateway + API JWT, owner-aware SQL, server-derived sub/sid, internal provider credentials.

****Security Misconfiguration:**** production Swagger off, bounded HTTP config, no server header, separate environment config.

****Supply Chain:**** central NuGet versions, explicit packages, CI restore/build/test. SBOM/vulnerability scanning remains open work.

****Cryptography:**** PBKDF2, salts, HMAC OTP, short JWT, secret store, no credential logging.

****Injection:**** parameterized SQL, typed DTOs, no user-controlled SQL fragments.

****Sensitive Business Flows:**** double-entry, durable idempotency, server-side fraud and fail-closed provider behavior.

****Authentication Failures:**** JWT + sessions + refresh rotation + logout + OTP + rate limits.

****Integrity Failures:**** DB constraints, append-only/reversal model and atomic posting.

****Resource Consumption:**** layered rate/body/header/connection/time limits.

****SSRF/Unsafe APIs:**** provider URLs are server-owned configuration and external results remain behind adapters/validation.

Configuration model

Standard .NET precedence:

- 1 appsettings.json;
- 2 appsettings.{Environment}.json;
- 3 environment variables / secret injection;
- 4 explicit providers added later.

Operational values such as network/pool/timeout/rate settings are configurable. Crypto algorithms, double-entry invariants and persistence semantics are not runtime switches.

First Run Happy Path: Registration to Wallet Transfer

This document is for someone seeing the project for the first time. All normal client calls go through YARP Gateway.

Base URL

```
{{gateway}} = http://localhost:8080
```

FinWallet.Api on :8081 should not be called directly for normal client flows.

Example variables:

```
{{tokenA}} Customer A JWT
{{tokenB}} Customer B JWT
{{walletA}} Customer A TRY wallet ID
{{walletB}} Customer B TRY wallet ID
```

1. Register Customer A

```
POST {{gateway}}/api/v1/auth/register
Content-Type: application/json
X-Correlation-Id: demo-register-a
```

```
{
  "countryCode": "TR",
  "phoneNumber": "+905321111111",
  "email": "customer.a@example.test",
  "password": "Example-Password-A-123!"
}
```

Expected: HTTP 202 with registration pending response. Save customerId.

****OTP note:**** FinWallet never returns OTP in the registration response. FakeCommunication receives the raw OTP as the simulated SMS body. Read it only through local test/debug instrumentation; do not add a public OTP-leak endpoint.

2. Verify Customer A OTP

```
POST {{gateway}}/api/v1/auth/registration/verify
Content-Type: application/json
```

```
{
  "customerId": "<customer-a-id>",
  "code": "123456"
}
```

Expected: HTTP 200 and registration verified.

3. Login Customer A

```
POST {{gateway}}/api/v1/auth/login
Content-Type: application/json
```

```
{
  "phoneNumber": "+905321111111",
  "password": "Example-Password-A-123!",
  "deviceId": "demo-device-a"
}
```

Store response accessToken as tokenA. Never log the token.

4. Create Customer A TRY wallet

```
POST {{gateway}}/api/v1/wallets
Authorization: Bearer {{tokenA}}
Content-Type: application/json
```



```
{ "currency": "TRY" }
```

First call: HTTP 201 WALLET_CREATED; repeat for the same customer/currency: HTTP 200 WALLET_EXISTS. Store walletId as walletA. Initial available/blocked balances are zero.

5. Open Customer A external bank account

Not required for an internal wallet transfer but part of the intended onboarding story.

```
POST {{gateway}}/api/v1/bank-accounts
Authorization: Bearer {{tokenA}}
Content-Type: application/json

{ "walletId": "{{walletA}}" }
```

Provider pending may return HTTP 202. Retrying must not create a duplicate provider account because the durable BankAccount ID produces a deterministic provider request key.

6. Register + verify + login Customer B

Repeat steps 1-3 with different phone/email/device. Example phone: +905322222222. Store access token as tokenB.

7. Create Customer B TRY wallet

```
POST {{gateway}}/api/v1/wallets
Authorization: Bearer {{tokenB}}
Content-Type: application/json

{ "currency": "TRY" }
```

Store wallet ID as walletB.

8. Verify wallet list

```
GET {{gateway}}/api/v1/wallets
Authorization: Bearer {{tokenA}}
```

If JWT is missing/invalid, Gateway rejects the request before forwarding it to the backend.

9. Funding prerequisite — current gap

A new wallet starts at zero balance. A public BankDeposit/funding endpoint **does not exist yet**, so public APIs alone cannot complete register -> fund -> successful transfer.

Do not do this:

```
UPDATE Wallets SET AvailableBalance = ...
```

It creates money outside the Ledger and breaks reconciliation.

For transfer demos/tests, a controlled integration fixture must atomically create balance + FinancialTransaction + balanced LedgerJournal/Entries. BankDeposit should be the next real feature.

10. Wallet transfer

After walletA has been validly funded:

```
POST {{gateway}}/api/v1/transfers
Authorization: Bearer {{tokenA}}
Idempotency-Key: demo-transfer-a-to-b-0001
Content-Type: application/json
X-Correlation-Id: demo-transfer-a-to-b

{
  "sourceWalletId": "{{walletA}}",
  "destinationWalletId": "{{walletB}}",
  "amount": 125.50
}
```

Expected success: HTTP 200 WALLET_TRANSFER_COMPLETED, immutable transactionId, amount/currency/completedAt and wasReplay=false.

11. Safe replay

Send the same request with the same Idempotency-Key again. Expected:

- no second money movement;
- no second ledger posting;
- no second fraud evaluation for a completed replay;
- same transaction ID;
- WALLET_TRANSFER_REPLAYED;
- wasReplay=true.

Using the same key with a different amount/destination returns a conflict.

Gateway expectations

Request	Expected behavior
Register/verify/login/refresh without JWT	Gateway forwards
Wallet/bank-account/transfer without JWT	Gateway rejects
/providers/* without internal key	Gateway rejects
Backend business endpoint without downstream key	Destination rejects
Rate/body/header limits exceeded	Rejected before business processing

Swagger

```
http://localhost:8080/swagger Gateway
http://localhost:8081/swagger FinWallet.Api
http://localhost:8082/swagger FakeBank
http://localhost:8083/swagger FakeFraud
```

Normal business calls still go through Gateway :8080.

AI-Assisted Architecture Decision Narrative

1. Starting problem

The side project selected a financial combination rather than generic CRUD:

Digital Wallet + Double-entry Ledger + Fraud Detection + External Banking Integration

The goal was to exercise real backend concerns such as concurrency, idempotency, provider failures, accounting, session security, auditability and reconciliation.

2. Why a modular monolith?

Wallet, Ledger and Transaction participate in the same atomic money movement. Splitting them into microservices immediately would introduce distributed-consistency costs too early. Domain modules remain logically separated and deployment boundaries can be reconsidered later.

3. Why MSSQL as the financial source of truth?

The system needs ACID transactions, constraints, FKs, unique keys, explicit locking and deterministic reconciliation queries. Redis is useful for fast transient state but is not allowed to decide whether money exists.

4. Why authentication first?

Wallet, BankAccount and Transfer ownership require stable customer/session identity. Registration -> OTP -> password -> login -> JWT -> refresh/session therefore came before money endpoints.

5. Why JWT plus server-side sessions?

JWT authenticates requests but does not independently provide immediate revoke. A minimal sid claim plus durable CustomerSession provides revoke/device/session checks for high-risk flows.

6. Why separate Wallet and BankAccount?

Wallet is internal customer liability/balance state. BankAccount is an external-provider relationship. Their lifecycle and source of truth differ, so they are not one aggregate.

7. Why FakeBank as a separate API?

An HTTP boundary forces the project to handle actual integration concerns: timeout, 5xx, pending state, provider-generated identity, idempotency and polling. FinWallet never accesses provider storage directly.

8. Why durable state before bank HTTP?

Bank-account opening persists internal BankAccount(Opening), closes SQL work, then calls the provider. The durable internal ID generates a deterministic provider request key so lost responses do not create duplicate external accounts.

9. Why Ledger before the transfer endpoint?

Balance answers "how much exists now?" Ledger answers "why does that balance exist?". Money endpoints were not exposed before accounting invariants existed.

10. Why FinancialTransaction separate from LedgerJournal?

FinancialTransaction represents the business operation/lifecycle; LedgerJournal represents the accounting effect. Keeping them separate supports failure, reversal, idempotency resource identity and reconciliation.

11. Why an explicit transfer store?

SqlWalletTransferPostingStore owns idempotency, both wallet updates, FinancialTransaction and Ledger in one MSSQL transaction. Locking and commit order are correctness rules here, so explicit SQL is preferred over a generic repository abstraction.

12. Why durable idempotency in MSSQL?

Mobile timeout, Gateway timeout, duplicate clicks and retries can resend the same financial command. Final duplicate protection is not delegated to process cache or Redis. The identity uses Scope + CustomerId + Key plus a canonical payload hash.

13. Why fraud before the money SQL transaction?

External HTTP is slow and uncertain. Fraud is evaluated first; a short atomic financial SQL transaction starts only after final Allow. Required fraud unavailable => fail closed.

14. Why completed replay before fraud?

A transaction completed yesterday should not be denied today when the client simply retries the same request under changed fraud rules. Completed immutable results are replayed first.

15. Why controller-based APIs?

All HTTP services use controllers for consistent enterprise Web API conventions, attributes, Swagger and explicit action contracts. Minimal API conventions are not mixed into the same codebase.

16. Why is ServiceResult only an HTTP contract?

ServiceResult<T> standardizes client responses. Making Domain/Application depend on it would unnecessarily couple business objects to HTTP transport.

17. Why YARP after financial core?

Routing infrastructure cannot fix incorrect money logic. Gateway was added as the edge/platform boundary after auth, persistence, ledger and transfer correctness were established.

18. Why service-to-service through Gateway?

Provider adapters use /providers/*. Two credentials exist:

FinWallet -> Gateway	InternalServiceKey
Gateway -> Backend	DownstreamServiceKey

This makes Gateway an enforceable trust boundary rather than only a client convention.

19. Why Shared.Web?

Duplicating Swagger/rate/CORS/header code across seven Web APIs would cause configuration drift. FinWallet.Shared.Web centralizes hosting concerns without becoming a business framework.

20. Why are some values configurable and others not?

Addresses, timeouts, pools, rate limits, body/header/connection limits, CORS, Swagger and YARP destinations are operational and therefore configurable. HS256 algorithm, PBKDF2 V1 migration semantics, Debit=Credit, financial decimals and durable idempotency semantics remain correctness/security invariants rather than arbitrary runtime toggles.

21. Why not expand Redis into a general cache?

Current use focuses on OTP/transient state. Wallet balances and final idempotency truth remain in MSSQL to avoid unnecessary invalidation and correctness risks.

22. Why add mocks but not treat them as sufficient?

xUnit + Moq can verify Application orchestration calls. SQL Serializable/range-lock behavior and YARP route correctness require real infrastructure tests.

23. Implementation sequence

- scope/invariants
 - > architecture boundaries
 - > registration/auth/session
 - > communication/fraud providers
 - > wallet/ledger domain
 - > FakeBank + BankAccount
 - > persistence
 - > wallet APIs
 - > financial transaction/idempotency schema
 - > atomic transfer store
 - > fraud/session transfer orchestration
 - > YARP Gateway
 - > Shared.Web + Swagger/security
 - > config/performance tuning
 - > xUnit/Moq + CI tests
 - > bilingual/current documentation

24. Natural next order

- 1 BankDeposit;
- 2 BankWithdrawal + cutoff;
- 3 integration/concurrency tests;
- 4 Outbox/Inbox + notification;
- 5 FraudEvents/manual review;
- 6 transaction history;
- 7 refund/reversal;
- 8 reconciliation;
- 9 OpenTelemetry/masked logging/operations.

25. Core rule

Do not expose the easy endpoint before the hard invariant underneath it exists.

Transfer is not exposed before ledger/idempotency; bank flows should not be exposed before durable state; provider retries should not exist without idempotency.

MSSQL, Redis, HTTP and Gateway Performance Review

Summary

Performance changes must not weaken financial correctness. Operational tuning is configuration-driven where appropriate; accounting and locking invariants are not relaxed merely for benchmark numbers.

MSSQL connection pooling

SqlClient pooling is enabled through the connection string. Development baseline:

- Pooling=True;
- Min Pool Size=5;
- Max Pool Size=100;
- bounded connect timeout;
- Load Balance Timeout;
- application name.

SqlConnectionFactory creates short-lived logical connections; disposal returns physical connections to the pool. Production pool size should be tuned from measured database capacity.

Financial-transaction performance

Wallet transfer commits source/destination balances, idempotency, FinancialTransaction and Ledger in the same transaction. Lock duration matters, but not more than correctness. Wallet rows are locked in deterministic GUID order to reduce opposite-direction deadlock risk.

Isolation is not weakened solely to improve throughput.

Index approach

Existing unique/index structures support important hot paths. Speculative indexes on write-heavy financial tables may increase insert/update/log cost. New indexes should be driven by Query Store, execution plans, logical reads and measured p95/p99 evidence.

Monitor:

- pool exhaustion;
- login/transfer DB p95/p99;
- deadlocks;
- lock waits;
- top logical reads;
- Query Store regressions;
- transaction-log growth;
- index usage/fragmentation.

Redis

A single ConnectionMultiplexer remains singleton. Creating a multiplexer per request would be a socket/performance regression.

Configurable values:

- AbortOnConnectFail;
- connect retry;
- connect timeout;
- sync timeout;
- keep-alive;
- client name.

OTP fail-closed security is not weakened for performance. Redis is not a financial authority.

Monitor reconnects, latency, timeouts, CPU, memory, evictions, network and Lua latency.

HttpClient

Provider clients use HttpClientFactory + SocketsHttpHandler with:

- pooled connection lifetime;
- pooled idle timeout;
- max connections/server;
- GZip/Deflate/Brotli;
- provider-specific timeout;
- cookies disabled.

Provider calls go through YARP Gateway. Financial POSTs are not automatically retried without endpoint-specific idempotency guarantees.

YARP

Clusters use PowerOfTwoChoices. Production destination replicas can be added through configuration. Performance controls include:

- load balancing;
- active health;
- passive health for the main cluster;
- max destination connections;
- activity timeout;
- HTTP version policy;
- response-buffering policy;
- request-size limits before proxying.

Rate limiting

Gateway public limits are stricter than backend limits. Backend limits provide a second layer against bypass, misrouting and internal runaway traffic. Queue defaults to zero; fast 429 rejection during overload reduces memory pressure and tail latency compared with building a large in-memory queue.

Optimizations deliberately not made

- No Redis wallet balance as source of truth.
- No blind weakening of financial SQL isolation.
- No automatic retry for arbitrary financial POSTs.
- No authenticated financial-response caching.

- No unbounded DB/HTTP connection pool.
- No bypassing fraud checks for latency.
- No ledger denormalization that would break reconciliation.

Benchmark plan

Measure:

- 1 registration RPS;
- 2 login p50/p95/p99;
- 3 wallet-list p50/p95/p99;
- 4 transfer p50/p95/p99;
- 5 same-wallet concurrent transfer throughput;
- 6 Gateway overhead versus a controlled direct internal baseline;
- 7 Redis OTP latency;
- 8 SQL locks/deadlocks;
- 9 CPU/memory/GC/socket counts;
- 10 ledger/balance/idempotency reconciliation after load.

A benchmark result is invalid if final balances and ledger state do not reconcile.

Final Technical Review

Review summary

This document evaluates the current main architecture after the Gateway/security/platform merges. The latest full-solution CI validation completed restore, Release build with --warnaserror, and dotnet test successfully. This proves compilation and the current unit-test scope are green; it does not mean real-infrastructure concurrency testing is complete.

Strengths

- MSSQL is the financial source of truth; Redis is not an authority for money correctness.
- Wallet transfer commits balances, idempotency, transaction and ledger state in one SQL transaction.
- Double-entry Debit = Credit is validated both in Domain and persisted SQL.
- Durable idempotency prevents duplicate money movement.
- External HTTP runs outside the financial SQL transaction.
- Fraud risk signals are derived from server state rather than trusted from the client.
- JWT plus durable sessions provide revocation checks for high-risk operations.
- YARP Gateway provides public auth, internal routing, rate limiting, health and load-balancing boundaries.
- Backend/provider business endpoints require a downstream service credential to reduce direct-bypass risk.
- Swagger is standardized across all Web APIs and disabled by default in production.
- Central package management and warnings-as-errors are enabled.
- xUnit v3 + Moq exist and CI executes tests.

Status of the previous request

- YARP Gateway: ****completed****.
- Routing service traffic through Gateway: ****implemented****; FinWallet provider adapters use /providers/*.
- Gateway JWT enforcement: ****completed****.
- Load balancing / health / rate limiting / request limits: ****completed****.
- Swagger on all APIs: ****completed****.
- OWASP/API-abuse hardening: ****completed at application level****; volumetric DDoS still requires edge/ingress/WAF deployment controls.
- MSSQL/Redis/HttpClient/YARP performance review: ****completed****.
- Moving operational/tuning parameters into appsettings: ****completed****.
- Unit-test mock verification: previously missing; ****xUnit + Moq added****.
- Registration-to-transfer happy-path document: ****completed****, with the public-funding gap documented explicitly.
- AI architecture decision narrative: ****completed****.
- All maintained project documentation in TR+EN: ****completed****.

Values intentionally not moved into appsettings

The request for “all parameterized elements” was applied to operational and tuning values. The following deliberately remain non-runtime-switch invariants:

- JWT signing algorithm;
- PBKDF2 V1 scheme/work-factor migration semantics;
- double-entry equality;
- financial decimal invariants;
- durable idempotency semantics;
- transaction/locking correctness rules.

Making these arbitrary configuration switches could weaken security or make existing persisted data unverifiable.

Open technical debt

- 1 Public BankDeposit and BankWithdrawal.
- 2 Durable FraudEvents/manual review.
- 3 Outbox/Inbox and reliable post-commit notification.
- 4 ReconciliationRuns/ReconciliationIssues.
- 5 Transaction-history/read model.
- 6 Centralized masked structured logging + OpenTelemetry/alerting.
- 7 Real MSSQL/Redis/YARP integration and concurrency test suite.
- 8 Public logout/session-revoke endpoint.
- 9 Dependency vulnerability/SBOM pipeline.
- 10 Production ingress/WAF/network-policy/TLS deployment hardening.

Recommended priority

- 1 BankDeposit;
- 2 BankWithdrawal + cutoff;
- 3 integration/concurrency tests;
- 4 Outbox/Inbox + notification;
- 5 FraudEvents/manual review;
- 6 transaction history;
- 7 reconciliation;
- 8 telemetry/operations hardening.

Final assessment

The current system is a solid financial-core/gateway baseline for a side project. It should not be described as a production-ready banking platform until funding/withdrawal, reconciliation, observability and real-infrastructure concurrency tests are complete.

Docker Runbook

Purpose

This document is the operational runbook for starting the entire FinWallet solution with Docker Compose for the first time, validating service health, inspecting logs, managing MSSQL/Redis volumes, rebuilding individual services, and shutting the stack down safely.

Normal Docker topology:

```
Host
|
| http://localhost:8080
v
FinWallet.Gateway
|
+--> FinWallet.Api
|
+--> FakeBank.Api
+--> FakeFraud.Api
+--> FakeCutoff.Api
+--> FakeCampaign.Api
+--> FakeCommunication.Api

FinWallet.Api
|
+--> MSSQL
+--> Redis
+--> Gateway /providers/*
```

Only the Gateway is published to the host in the normal stack. MSSQL, Redis, FinWallet.Api and fake-provider ports stay inside Docker networks. `compose.debug.yml` publishes them only on 127.0.0.1 for local inspection/debugging.

Docker files

File	Purpose
<code>compose.yml</code>	Main Compose definition for the application and infrastructure services.
<code>compose.debug.yml</code>	Debug overlay exposing MSSQL, Redis, FinWallet.Api and provider ports on localhost only.
<code>compose.production.yml</code>	Production-like hardening overrides such as disabling Swagger and stronger restart policies.
<code>docker/Dockerfile.webapi</code>	Shared multi-stage .NET 8 image build for Gateway, FinWallet.Api and all fake Web APIs.
<code>docker/mssql/init-db.sh</code>	Waits for MSSQL and applies the 001, 002, 003 schema scripts with schema-version tracking.
<code>.env.example</code>	Template for local Compose environment variables; contains no real production secrets.
<code>.dockerignore</code>	Removes <code>bin/obj/log/test/docs/local-secret</code> content from the Docker build context.
<code>.gitignore</code>	Prevents build output, logs, secrets and local Docker state from entering Git.

Docker services

Service	Responsibility	Normal host port
gateway	YARP edge gateway, JWT/routing/rate limit/load balancing	8080
finwallet-api	Main FinWallet Web API	not published

Service	Responsibility	Normal host port
fake-bank	Bank simulator	not published
fake-fraud	Fraud simulator	not published
fake-cutoff	Cutoff/business-calendar simulator	not published
fake-campaign	Campaign simulator	not published
fake-communication	SMS/Email simulator	not published
mssql	Financial source of truth	not published
mssql-init	One-shot schema initialization container	none
redis	OTP/transient support state	not published

Persistent volumes

Volume	Content	Survives docker compose down?
finwallet_mssql_data	SQL Server database/log files	Yes
finwallet_mssql_backup	SQL backup area	Yes
finwallet_redis_data	Redis AOF/RDB persistence	Yes

Application containers are stateless. Application logs are not persisted to application-specific named volumes; they go to stdout/stderr and Docker log rotation is configured.

1. Verify prerequisites

```
docker --version
docker compose version
```

What it does:

- Confirms Docker Engine/Desktop CLI is available.
- Confirms the Compose v2 plugin is installed.
- If these fail, fix Docker before starting FinWallet.

On Windows Docker Desktop, use Linux containers.

2. Change to the repository root

```
cd finwallet
```

Run the following commands from the directory containing FinWallet.sln, compose.yml, and .env.example.

3. Create the local environment file

PowerShell:

```
Copy-Item .env.example .env
```

Bash:

```
cp .env.example .env
```

What it does:

- Creates a local .env from the version-controlled template.
- .env is ignored by Git and must not be committed.

Replace at least these values with strong local values:

```
MSSQL_SA_PASSWORD
```

```
REDIS_PASSWORD
JWT_SIGNING_KEY
REGISTRATION_OTP_PEPPER
INTERNAL_SERVICE_KEY
DOWNSTREAM_SERVICE_KEY
```

Do not use .env as the production secret-management mechanism. Use orchestrator/secret-store injection in production.

4. Validate Compose before starting containers

```
docker compose --env-file .env config --quiet
```

What it does:

- Resolves environment substitutions.
- Validates YAML/Compose syntax.
- Fails early when a required environment variable is missing.
- Does not start containers or modify data.

To print the resolved model:

```
docker compose --env-file .env config
```

Warning: resolved output can contain secrets. Do not redirect it into shared CI logs or files.

List services:

```
docker compose --env-file .env config --services
```

5. Build images

```
docker compose --env-file .env build --pull
```

What it does:

- Builds Gateway, FinWallet.Api and fake API images using the multi-stage Dockerfile.
- --pull checks for a newer base image matching the configured tag.
- NuGet restore uses the BuildKit cache.
- Does not start containers.

For a fully clean build:

```
docker compose --env-file .env build --pull --no-cache
```

This is slower and should not be the default development workflow.

6. Start the entire system

```
docker compose --env-file .env up -d --build
```

What it does:

- 1 Builds/pulls required images.
- 2 Creates finwallet-data and finwallet-backend networks.
- 3 Creates or reuses MSSQL and Redis named volumes.
- 4 Waits for MSSQL health checks.
- 5 Runs mssql-init, which creates the FinWallet database and unapplied schema versions.
- 6 Starts FinWallet.Api after Redis is healthy and DB initialization completes.
- 7 Starts fake providers and Gateway.
- 8 -d runs the stack in detached mode.

The first build/pull takes longer than subsequent starts.

7. Inspect container status

```
docker compose --env-file .env ps
```

Expected state:

- mssql: Up / healthy
- redis: Up / healthy
- mssql-init: Exited (0)
- API and Gateway services: Up

mssql-init being Exited (0) is expected because it is a one-shot migration job.

Docker-level view:

```
docker ps --filter name=finwallet
```

8. Verify the Gateway health endpoint

```
curl http://localhost:8080/health/live
```

Windows PowerShell:

```
Invoke-RestMethod http://localhost:8080/health/live
```

If this responds, the host-to-Gateway path is available.

When Swagger is enabled in Docker/Development:

```
http://localhost:8080/swagger
```

Normal client API calls should use Gateway at <http://localhost:8080>.

9. Follow logs

Entire stack:

```
docker compose --env-file .env logs -f
```

Gateway:

```
docker compose --env-file .env logs -f gateway
```

FinWallet API:

```
docker compose --env-file .env logs -f finwallet-api
```

MSSQL:

```
docker compose --env-file .env logs -f mssql
```

Redis:

```
docker compose --env-file .env logs -f redis
```

Last 200 lines:

```
docker compose --env-file .env logs --tail 200 gateway finwallet-api
```

-f follows new log entries. Ctrl+C exits log following only; detached containers continue running.

Docker json-file log rotation is configured. Application logs are sent to stdout/stderr rather than durable files inside application containers.

10. Verify MSSQL

```
docker compose --env-file .env exec mssql bash -lc '
SQLCMD=$(command -v /opt/mssql-tools18/bin/sqlcmd || command -v /opt/mssql-tools/bin/sqlcmd)
&&
```

```
$SQLCMD -S localhost -U sa -P "$MSSQL_SA_PASSWORD" -C -d FinWallet -Q "SELECT DB_NAME() AS
DatabaseName; SELECT Version, AppliedAt FROM dbo.SchemaVersions ORDER BY Version;"
```

This:

- connects from inside the SQL Server container;
- confirms the FinWallet database exists;
- lists applied 001, 002, 003 schema versions.

Re-run the migration check manually with:

```
docker compose --env-file .env run --rm mssql-init
```

Already-recorded schema versions are skipped.

11. Verify Redis

```
docker compose --env-file .env exec redis sh -lc 'redis-cli -a "$REDIS_PASSWORD"
--no-auth-warning ping'
```

Expected:

PONG

Persistence details:

```
docker compose --env-file .env exec redis sh -lc 'redis-cli -a "$REDIS_PASSWORD"
--no-auth-warning INFO persistence'
```

12. Publish debug ports

The normal stack intentionally does not publish backend services. For local debugging:

```
docker compose --env-file .env -f compose.yml -f compose.debug.yml up -d --build
```

Loopback ports become:

Gateway	8080
FinWallet.Api	8081
FakeBank.Api	8082
FakeFraud.Api	8083
FakeCutoff.Api	8084
FakeCampaign.Api	8085
FakeCommunication.Api	8086
MSSQL	1433
Redis	6379

The overlay binds debug ports to 127.0.0.1, not the LAN.

Use Gateway for normal integration calls. Direct backend ports are for inspection/debugging and downstream service-key checks remain active.

13. Rebuild/restart one service

If FinWallet.Api code changed:

```
docker compose --env-file .env build finwallet-api
docker compose --env-file .env up -d --no-deps finwallet-api
```

What it does:

- rebuilds only the FinWallet.Api image;
- recreates only that service;
- --no-deps avoids recreating MSSQL/Redis/Gateway.

For Gateway:

```
docker compose --env-file .env build gateway
docker compose --env-file .env up -d --no-deps gateway
```

Restart without rebuilding:

```
docker compose --env-file .env restart gateway
```

14. Open a shell inside a service

```
docker compose --env-file .env exec finwallet-api /bin/sh
```

The runtime image executes the service as the non-root app user. Do not expect root privileges.

Avoid printing the full environment because it contains secrets.

15. Monitor resource usage

```
docker stats
```

It shows CPU, memory, network I/O, block I/O and PID counts.

Compose sets CPU, memory and PID limits for application containers. These are local safety baselines, not production capacity numbers.

16. Inspect volumes

```
docker volume ls --filter name=finwallet
```

Inspect individually:

```
docker volume inspect finwallet_mssql_data
docker volume inspect finwallet_mssql_backup
docker volume inspect finwallet_redis_data
```

Normal restart/recreate operations keep these volumes.

17. Create an SQL backup

Create a backup:

```
docker compose --env-file .env exec mssql bash -lc '
SQLCMD=$(command -v /opt/mssql-tools18/bin/sqlcmd || command -v /opt/mssql-tools/bin/sqlcmd)
&&
$SQLCMD -S localhost -U sa -P "$MSSQL_SA_PASSWORD" -C -Q "BACKUP DATABASE [FinWallet] TO
    DISK = N''''/var/opt/mssql/backup/FinWallet.bak'''' WITH INIT, CHECKSUM"
'
```

The backup remains in the mssql_backup named volume.

Get the container ID:

```
docker compose ps -q mssql
```

Copy to the host in Bash:

```
docker cp "$(docker compose ps -q mssql):/var/opt/mssql/backup/FinWallet.bak"
./FinWallet.bak
```

Do not commit backups to Git.

18. Request a Redis snapshot

```
docker compose --env-file .env exec redis sh -lc 'redis-cli -a "$REDIS_PASSWORD"
--no-auth-warning BGSAVE'
```

This asks Redis to create a background snapshot. Redis is still not the financial source of truth and this is not a substitute for MSSQL backup.

19. Stop and start without deleting containers

Stop:

```
docker compose --env-file .env stop
```

Start:


```
docker compose --env-file .env start
```

stop does not remove containers or volumes.

20. Shut down while preserving data

```
docker compose --env-file .env down
```

It:

- removes Compose containers;
- removes Compose networks;
- preserves named volumes;
- allows the next up to reuse MSSQL/Redis data.

21. Full reset - DESTRUCTIVE

```
docker compose --env-file .env down -v --remove-orphans
```

It removes:

- containers;
- networks;
- mssql_data, mssql_backup, redis_data named volumes.

Result: local database, ledger, customer/auth data, Redis state and SQL backup volume are deleted.

Use this only for a deliberate clean local reset.

22. Reset Redis only

```
docker compose --env-file .env down
docker volume rm finwallet_redis_data
docker compose --env-file .env up -d
```

MSSQL data remains intact.

23. Reset MSSQL only

```
docker compose --env-file .env down
docker volume rm finwallet_mssql_data finwallet_mssql_backup
docker compose --env-file .env up -d
```

This permanently removes local financial data. Do not use outside disposable local development.

24. Inspect and clean Docker disk usage

```
docker system df
```

Build cache cleanup:

```
docker builder prune
```

Unused image cleanup:

```
docker image prune
```

docker system prune -a --volumes is much more destructive and can remove resources used by other projects; it should not be a routine command.

25. Use the production-like overlay

```
docker compose \
  --env-file .env \
  -f compose.yml \
  -f compose.production.yml \
  up -d --build
```

The overlay:

- sets ASPNETCORE_ENVIRONMENT=Production;
- disables Swagger;
- strengthens restart policy;
- enables HSTS configuration.

This is not a replacement for a real production platform. Real production also requires TLS ingress, external secret management, WAF/DDoS controls, image digest pinning, vulnerability scanning, backup/restore procedures, monitoring/alerting and network policy.

26. Update the running stack after code changes

```
git pull
docker compose --env-file .env build --pull
docker compose --env-file .env up -d
```

When a new schema script is added, mssql-init must also be updated to recognize the new migration. Merely adding a SQL file is not sufficient.

27. Common problems

Port 8080 already in use

```
docker compose --env-file .env ps
```

Change .env:

```
GATEWAY_PORT=8090
```

Run up -d again. Gateway becomes http://localhost:8090.

MSSQL unhealthy

```
docker compose --env-file .env logs --tail 300 mssql
```

Check:

- SA password complexity;
- Docker memory availability;
- damaged/incompatible volume;
- restart loop.

mssql-init failed

```
docker compose --env-file .env logs mssql-init
```

Fix the schema error, then run:

```
docker compose --env-file .env run --rm mssql-init
```

Redis unhealthy

```
docker compose --env-file .env logs redis
docker compose --env-file .env exec redis sh -lc 'redis-cli -a "$REDIS_PASSWORD"
--no-auth-warning ping'
```

Gateway returns 502/503

```
docker compose --env-file .env logs --tail 200 gateway finwallet-api fake-bank fake-fraud
fake-cutoff fake-campaign fake-communication
```

Check:

- downstream service is Up;

- Docker service DNS address is correct;
- internal/downstream keys match;
- Gateway health checking has not marked the destination unhealthy.

Required secret is missing

Compose fails with a message similar to:

```
required variable ... is missing a value
```

Confirm .env exists at repository root and required keys are non-empty.

28. Minimum daily development command set

First day:

```
cp .env.example .env
docker compose --env-file .env config --quiet
docker compose --env-file .env up -d --build
docker compose --env-file .env ps
docker compose --env-file .env logs -f gateway finwallet-api
```

At the end of the day, preserve data:

```
docker compose --env-file .env down
```

Next day:

```
docker compose --env-file .env up -d
docker compose --env-file .env ps
```

For a full disposable local reset:

```
docker compose --env-file .env down -v --remove-orphans
docker compose --env-file .env up -d --build
```

Remember: the reset command deletes local MSSQL and Redis data.